

QUESTION 3: HOUGH TRANSFORM IN NOISY IMAGES

Scenario

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results. Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

Answer

1. Problem Analysis

The Hough Transform is commonly used to detect straight lines in an image. It works by identifying edge points and converting them into a parameter space, where points belonging to the same line form strong peaks. However, when the image contains noise, unwanted pixels may also be detected as edges. These false edge points can create incorrect votes in the Hough space, leading to inaccurate line detection.

2. Effect of Noise

Noise can affect line detection in several ways:

- It produces unwanted or false edge points.
- It may result in the detection of incorrect lines.
- Extra and irrelevant lines may be detected.
- Actual lines may appear broken or fragmented.
- The Hough parameter space may contain false peaks.
- Overall line detection accuracy is reduced.

3. Recommended Improvements

Preprocessing:

Apply a Gaussian filter or Median filter before edge detection. These filters help reduce noise while preserving important image structures.

Edge Detection:

Use the Canny edge detector to obtain cleaner and more reliable edge points before applying the Hough Transform.

Parameter Selection:

The Hough Transform parameters should be selected carefully:

- Threshold: Controls the minimum number of votes required to detect a line. Increasing the threshold can reduce false lines.

- Minimum Line Length: Helps remove very short and insignificant line segments.
- Maximum Line Gap: Controls the maximum gap allowed between connected line segments.

4. Conclusion

Noise can significantly reduce the accuracy of the Hough Transform by creating false edge points and incorrect votes. Applying noise reduction, effective edge detection, and suitable Hough parameters can reduce false detections and improve the accuracy and reliability of line detection.

QUESTION 4: FEATURE MATCHING UNDER TRANSFORMATIONS

Scenario

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale-invariant feature matching works effectively. Analyze why scale-invariant methods perform better in this scenario.

Answer

1. Problem Analysis

Feature matching is used to identify the same points or regions in two images. Traditional feature matching methods may depend on the size and orientation of the object. When the same object appears larger, smaller, rotated, or viewed from a different orientation, the feature locations and descriptions may change. This makes traditional methods less reliable.

2. Effect of Scale and Orientation

Changes in scale and orientation can cause:

- The same feature to appear at different sizes.
- Feature positions and directions to change.
- Traditional descriptors to produce different feature values.
- Incorrect matches or fewer matching points.
- Reduced matching accuracy.

3. Why Scale-Invariant Methods Perform Better

Scale-invariant methods such as **SIFT (Scale-Invariant Feature Transform)** are designed to detect and describe features regardless of changes in scale and orientation.

- **Scale Invariance:** Features are detected at different image sizes, allowing the same object to be recognized even when it is enlarged or reduced.
- **Rotation Invariance:** Feature descriptors are assigned a dominant orientation, making them less affected by image rotation.
- **Robust Feature Description:** SIFT describes the local region around a feature using distinctive patterns, which helps identify corresponding points.
- **Better Matching:** The descriptors remain relatively consistent even when the object undergoes scale or rotation changes.

4. Conclusion

Scale-invariant feature matching performs better because it is designed to handle changes in **scale and orientation**. Traditional methods are more sensitive to these transformations, while methods such as SIFT provide more stable and reliable feature descriptions. Therefore, scale-invariant techniques result in **higher matching accuracy and fewer incorrect matches**.

QUESTION 6: PREPROCESSING IMPACT ON EDGE DETECTION

Scenario

A system performs edge detection on raw images containing noise and uneven illumination, resulting in fragmented edges. Analyze the problem and explain how preprocessing improves edge detection accuracy and continuity.

Answer

1. Problem Analysis

Edge detection identifies boundaries where there is a significant change in image intensity. When raw images contain noise and uneven illumination, these intensity changes become irregular. As a result, the edge detector may identify unwanted edges or miss important boundaries, producing fragmented and incomplete edges.

2. Effect of Noise and Uneven Illumination

Noise and uneven lighting can cause:

- False edges due to random intensity variations.
- Important edges to become weak or unclear.
- Edges to appear broken or fragmented.
- Different parts of the same object to have different intensity levels.

- Reduced accuracy and continuity of detected edges.

3. Role of Preprocessing

Preprocessing improves edge detection by making the image cleaner and more uniform before applying the edge detection algorithm.

Noise

Removal:

Gaussian or Median filtering can reduce unwanted noise and prevent false edges.

Illumination

Correction:

Intensity normalization or background correction can reduce the effect of uneven lighting and make object boundaries more consistent.

Contrast

Enhancement:

Histogram Equalization or CLAHE can improve the contrast between objects and their backgrounds, making weak edges easier to detect.

Edge

Detection:

After preprocessing, methods such as Canny or Sobel can detect cleaner and more continuous edges.

4. Conclusion

Preprocessing improves edge detection by reducing noise, correcting uneven illumination, and enhancing important intensity differences. As a result, false edges are reduced, weak boundaries become clearer, and detected edges become **more accurate, connected, and continuous**.

QUESTION 10: PCA IN FACE RECOGNITION

Scenario

A face recognition system processes high-dimensional image data, resulting in slow performance. Analyze how PCA helps in reducing dimensionality while preserving important features.

Answer

1. Problem Analysis

Face images contain a large number of pixels, making them high-dimensional data. Processing all these pixels requires more memory and computational time. This can make the face recognition system slower and less efficient.

2. Problems with High-Dimensional Data

High-dimensional face images can cause:

- Increased processing time.
- Higher memory requirements.
- More complex computations.
- Difficulty in identifying the most important facial features.
- Reduced efficiency of the recognition system.

3. How PCA Helps

Principal Component Analysis (PCA) reduces the number of dimensions while retaining the most important information from the original image.

- PCA identifies the directions with the highest variation in the face images.
- These directions are called **principal components**.
- The original high-dimensional images are projected onto a smaller number of principal components.
- Important facial patterns such as the shape of the eyes, nose, mouth, and overall face structure are largely preserved.
- Less important and redundant information is removed.

4. Benefits of PCA

- Reduces the dimensionality of face images.
- Decreases memory usage.
- Speeds up face recognition and matching.
- Removes redundant information.
- Makes the system more computationally efficient.

5. Conclusion

PCA helps face recognition systems by converting high-dimensional face images into a smaller set of meaningful features. It preserves the major variations and important facial characteristics while removing unnecessary information. Therefore, PCA makes face recognition **faster, more efficient, and easier to process**.

QUESTION 15: COMBINED FEATURE DETECTION AND MATCHING

Scenario

A system detects features correctly but fails during matching due to incorrect descriptor selection. Analyze the issue and explain how appropriate feature descriptors improve matching performance.

Answer

1. Problem Analysis

Feature detection identifies important points such as corners and distinctive regions in an image. However, detecting features alone is not sufficient for successful matching. A descriptor is required to represent the local information around each detected feature. If an unsuitable descriptor is selected, the system may not correctly identify corresponding features between two images.

2. Effect of Incorrect Descriptor Selection

An inappropriate descriptor can cause:

- Incorrect feature matches.
- Fewer correct matches.
- Increased false matches.
- Poor performance under changes in scale or rotation.
- Reduced matching accuracy.
- Difficulty in handling changes in lighting or image appearance.

3. Role of Appropriate Feature Descriptors

Feature descriptors provide a numerical representation of the region surrounding each detected feature. Choosing a suitable descriptor makes the features easier to compare between images.

- **SIFT:** Provides strong performance under scale and rotation changes.
- **SURF:** Provides fast and robust feature description with good scale and rotation handling.
- **ORB:** Provides efficient and fast matching, making it suitable for real-time applications.
- **BRIEF:** Provides a compact binary descriptor for fast comparison.

The descriptor should be selected based on the type of transformation, image conditions, and computational requirements.

4. How Descriptors Improve Matching

An appropriate descriptor:

- Provides distinctive information about each feature.
- Makes corresponding features easier to identify.
- Reduces false matches.
- Improves robustness to scale, rotation, and illumination changes.
- Increases matching accuracy and reliability.

5. Conclusion

Correct feature detection must be followed by appropriate descriptor selection for successful feature matching. A suitable descriptor represents important local characteristics and allows corresponding features to be identified accurately. Therefore, selecting a descriptor such as **SIFT**, **SURF**, or **ORB** according to the application can significantly improve **matching accuracy, robustness, and performance**.